Lab1 实验报告：Data Lab

2026 年 10 月

Lab1 实验报告：Data Lab

一、实验结果

1.1 运算符检查

使用 `check_ops.py` 检查所有函数的运算符使用情况，19 个函数全部通过检查。

```
liu@LAPTOP-R7CO7ULB:/mnt/c/Users/liu/DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 9/20 operators
swapNibblePairs: 18/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 10/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 31/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 33/52 operators
floatScaleThreeHalves: 36/60 operators
floatRoundEven: 27/65 operators
float_i2f: 31/40 operators
bitCount: 27/40 operators
bitReverse: 33/34 operators
All 19 functions passed operator checks.
```

Figure 1: check_ops 结果

1.2 功能测试

使用 `btest` 测试所有函数的正确性，总分 110/110，所有函数测试通过，错误数为 0。

```
liu@LAPTOP-R7CO7ULB:/mnt/c/Users/liu/DataLab$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

Figure 2: btest 结果

二、各函数实现思路

P1: signMask（Rating: 1）

返回最高位为 1 的掩码（0x80000000）。直接将 1 左移 31 位即可。

```
return 1 << 31;
```

P2: bitXor（Rating: 2）

用 `~` 和 `&` 实现异或。根据德摩根定律，`x ^ y = ~(~x & ~y) & ~(x & y)`。前半部分 `~(~x & ~y)` 等价于 `x | y`，后半部分 `~(x & y)` 保留至少一个为 0 的位，两者相与即为异或。

P3: negativePart（Rating: 3）

当 x 为负数时返回 -x，否则返回 0。先通过算术右移 31 位得到符号掩码（负数为全 1，非负为全 0），再计算 `-x = ~x + 1`，最后用掩码筛选。

```
int mask = x >> 31;
int negX = ~x + 1;
return negX & mask;
```

P4: copyByteWithin（Rating: 4）

将 x 的第 src 字节复制到第 dst 字节。先提取源字节 `(x >> (src*8)) & 0xFF`，再构造目标位置的掩码清除原字节，最后将源字节放到目标位置。

P5: logicalShift（Rating: 4）

实现逻辑右移。算术右移后，高位会补符号位，需要把高位的 1 清零。构造一个低 `32-n` 位为 1 的掩码，与右移结果相与即可。掩码通过 `~((~0 << (31-n)) << 1)` 生成。

P6: swapNibblePairs（Rating: 4）

交换每个字节内的高低 4 位。构造两个掩码分别提取所有字节的低 4 位和高 4 位，低 4 位左移 4 位，高 4 位右移 4 位，再相或。由于常量限制（0-255），掩码通过 `0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24)` 的方式构造。

P7: secondLowestZeroBit（Rating: 4）

返回第二个最低 0 位的位置掩码。先找第一个 0 位：`~x & (x+1)` 利用补码性质提取最低 0 位。然后将该位设为 1（`x | firstZero`），再对新的数执行同样的操作找第二个 0 位。

P8: oddParity（Rating: 5）

返回 x 的奇校验位（1 的个数为偶数返回 1，否则返回 0）。通过逐次折半异或将所有位的信息压缩到最低位：先异或高 16 位和低 16 位，再异或高 8 位和低 8 位，依次类推到 1 位。最后 `(x & 1) ^ 1` 取反得到奇校验。

P9: rotateRightBits（Rating: 5）

循环右移 n 位。右移部分用逻辑右移并掩码保留低 `32-n` 位，左移部分直接左移 `32-n` 位，两者相或。

P10: roundEvenPow2（Rating: 5）

将非负数 x 舍入到最近的 2^n 的倍数，恰好在一半时向偶数方向舍入。先计算余数 `rem = x & mask`（mask 为低 n 位全 1），加上 `2^(n-1)` 后清零低 n 位得到初步舍入结果。如果余数恰好等于 `2^(n-1)` 且商为奇数，则减 1 使商变为偶数。

P11: midpointTowardFirst（Rating: 5）

计算 (x+y)/2 且不溢出，中点恰好为半整数时向 x 方向取整。利用公式 `(x+y)/2 = (x&y) + (x^y)/2` 避免溢出：`x&y` 是公共部分，`x^y` 是差异部分，差异部分右移 1 位相当于除以 2。如果差异部分最低位为 1（即存在 0.5），且 x > y，则结果加 1 使中点向 x 靠近。

P12: isBetweenEitherOrder（Rating: 7）

判断 x 是否在 a 和 b 之间（a、b 顺序不限）。分别判断 x 是否小于 a 和 x 是否小于 b，考虑溢出情况后用安全的方式计算大小关系。如果 x 小于其中一个但不小于另一个，说明 x 在两者之间。另外 x 等于 a 或 b 时也返回 1。

P13: mul5Sat（Rating: 7）

计算 x*5，溢出时饱和到 INT_MAX 或 INT_MIN。先算 `4x + x`，检测左移 2 位是否溢出（高 3 位与符号位不一致）以及加法是否溢出（操作数和结果符号不同）。溢出时根据 x 的符号返回 INT_MAX 或 INT_MIN，未溢出时返回正常结果。

P14: classifyAdd3（Rating: 7）

判断 x+y+z 的精确数学和是否超出 int 范围。分两步计算：先算 `s = x + y`，检测是否溢出；再算 `total = s + z`，检测是否溢出。两次溢出检测中，如果任何一次正溢出则返回 1，任何一次负溢出则返回 -1，否则返回 0。

P15: floatScaleThreeHalves（Rating: 7）

实现浮点数乘以 3/2。分三种情况处理:

- NaN/Inf（exp == 0xFF）：直接返回原值。
- 非规格化数（exp == 0）：尾数乘以 3 再除以 2，按 round-to-nearest-even 舍入。
- 规格化数：将隐含的 1 加入尾数得到完整有效数字，乘以 3。如果结果超过 2（即最高位进位），则右移 2 位并将指数加 1；否则右移 1 位。最后检查指数是否溢出到 Inf。

P16: floatRoundEven（Rating: 10）

将浮点数舍入到最近的整数，半值情况向偶数舍入。根据指数 e 分情况:

- exp < 127（值小于 1）：如果 exp == 126 且尾数非零，结果为 ±1.0；否则为 ±0。
- e >= 23：值已经是整数，直接返回。

- 其他情况：根据 `23 - e` 确定需要舍入的位数，提取余数部分，与半值比较，半值时检查整数部分的奇偶性决定是否进位。

P17: float_i2f（Rating: 10）

将 int 转换为 float 的位表示。先处理符号和特殊情况（x == 0）。找到最高位 1 的位置 msb，如果 msb <= 23 则直接左移对齐；如果 msb > 23 则需要右移截断，截断部分按 round-to-nearest-even 舍入。舍入后如果有效数字进位导致超出 23 位，需要再右移 1 位并将指数加 1。

P18: bitCount（Rating: 10）

统计 x 中 1 的个数。使用分治法（population count 算法）：先用掩码 0x5555…将相邻 1 位相加，再用 0x3333…将相邻 2 位相加，依次合并到 4 位、8 位、16 位，最终得到 1 的总数。掩码通过移位和或运算从 8 位常量构造。

P19: bitReverse（Rating: 10）

反转 32 位整数的所有位。使用分治交换法：先交换相邻 1 位，再交换相邻 2 位，依次交换 4 位、8 位、16 位。每一步用掩码分离奇偶位，分别移位后相或。掩码构造方式与 bitCount 类似。

三、参考资料

1. Computer Systems: A Programmer’s Perspective (CS:APP), Chapter 2 —整数和浮点数的位级表示、补码运算、IEEE 754 浮点数标准。
2. Data Lab 实验讲义—各函数的约束条件、允许使用的运算符、最大运算符数量。
3. “Bit Twiddling Hacks”(Sean Eron Anderson) —位操作技巧，包括 population count、位反转等算法的参考。

四、实验总结

这次实验一共 19 个函数，涵盖了整数位操作和浮点数位级表示两个方面。前面的整数题主要考察对补码、移位、掩码的理解，后面的浮点数题需要熟悉 IEEE 754 的编码方式。

整数部分比较难的是 midpointTowardFirst 和 classifyAdd3，需要在不溢出的前提下完成计算，关键是找到合适的数学等价变换。浮点数部分主要是 floatRoundEven 和 float_i2f，需要处理舍入和边界情况，逻辑相对复杂但思路比较直接。

通过这次实验，对计算机中数据的底层表示有了更具体的认识，尤其是浮点数的舍入规则和溢出处理，之前只在课本上看到过，这次亲手实现了一遍印象比较深。